TDD Sections to Address

Dr. Richard B. Johnson
UNCG Game Development Club

TDD Sections to Address

- To achieve a truly complete and robust document that guides the development team through the "how-to," the focus should shift to filling out the remaining sections and providing technical blueprints for the central game mechanics.
- **Core Systems Design**

  This section is currently strong on high-level goals but lacks the technical blueprints needed by programmers. Since these are the game's pillars, they must be fully defined.
- **Combat and AI Design**

  While the boss logic is outlined well, the foundation for all enemies needs explicit definition.
- **Technical Polish and Utilities**

  These sections are often overlooked but are critical for professionalism and performance.

Angel (Player) System: State Machine Diagram

- **What to add:** A flow chart or state diagram for the player's **Movement Model**.
- **Why:** A visual diagram is essential for defining the logic and transitions between states like **Idle** ↔ **Walk** ↔ **Run** ↔ **Dash** ↔ **Jump**. This also needs to explicitly show the transitions to/from the **Turn-Based Combat** state. This prevents bugs and ensures smooth controls.

- **Question 1:** Can you provide a flow chart detailing the transitions between the core Player movement states (Idle, Walk, Run, Dash, Jump, Interact, etc.) and the single entry / exit point for the Turn-Based state?

- **Question 2:** What is the mechanism for persisting the player's post-boss Health / Damage choices across new Rogue-lite cycles?

Memory and Ability Progression System: Data Schemas

- **What to add:** The final C# or GDScript structure (the class definition) for the **MemoryFragment** and the specific conditions that trigger ability injection in Phase 2 (**Parry**) and Phase 3 (**Riposte**).
- **Why:** Providing the exact data model and serialization format upfront is mandatory for the data management and UI teams.

- **Question 3:** What is the final, standardized class structure (fields and data types) for the **MemoryFragment** data model?

- **Question 4:** What are the exact in-game acquisition conditions that will trigger the dynamic injection of the **Parry** (Phase 2) and **Riposte** (Phase 3) abilities into the player's state?

Combat and AI Design
Enemy AI Framework: Base State Machine

- **What to add:** A visual **Finite State Machine (FSM)** diagram detailing the flow of the base AI between states: **Idle** ↔ **Patrol** ↔ **Chase** ↔ **Combat** ↔ **Retreat**.
- **Why:** This diagram is the technical blueprint for the **T2.4 Enemy AI State Machine** chunked task. It visually communicates how all standard enemies perceive the player and transition behavior, which is a major system dependency.

- **Question 6:** Produce a Finite State Machine (FSM) diagram showing the flow, states, and explicit trigger conditions between **Idle, Patrol, Chase, Combat, and Retreat** for the base Enemy AI.

Audio System: Technical Specification

- **What to add:** Define technical specifications for audio assets, such as the required **file formats** (e.g., OGG for music, WAV for short SFX), the **mixing pipeline** (e.g., separate audio buses for Music, SFX, and UI), and the specific **Dynamic Music** transition triggers.
- **Why:** This guides the technical sound designer and prevents the ingestion of incompatible or non-optimized asset files later in the pipeline.

- **Question 7:** What are the mandatory file formats, bitrates, and required audio buses (e.g., Music, SFX, UI) that will govern the mixing pipeline?

- **Question 8:** What are the specific in-game events (e.g., proximity trigger, Turn-BasedState load) that will trigger the transitions for the Dynamic Music System?

Data Management: Persistence Protocol

- **What to add:** Detail the technical protocol for **save files**. Specify the required **serialization format** (e.g., JSON or Godot's binary Resource format) and the exact data fields that must be persisted across **Rogue-lite runs** (e.g., Memory/Identity progress vs. temporary run data).
- **Why:** This is a high-risk area for data loss. Defining the protocol prevents bugs related to persistence and ensures the **T2.2 Rogue-lite Persistence** task is built on a solid foundation.

- **Question 9:** Which serialization format (e.g., JSON, binary) will be used for the save files, and which data fields are permanently persisted versus those that are reset during a Rogue-lite run?